Symmetry, rank, and solution compression:

towards structural algorithms for SAT and HAMPATH

Oscar Riveros

Santiago, Chile

Working version for discussion — October 30, 2026

Observation: this manuscript was prepared by the author with the collaboration of DeepSeek V4.1 Flash 1017 via Kilo Code and ChatGPT (GPT-6). Final mathematical and editorial responsibility rests with the author.

Abstract

We study compressed representations of solution families for SAT and HAMPATH, with emphasis on symmetries, tensor rank, and composition of subproblems. First, we formalize the quotient of assignments by automorphism groups, extend fixed-point counting to signed permutations, and show that compression by orbits does not by itself avoid evaluating all representatives. Second, we express HAMPATH as an orbit–subcube intersection and analyze why exact isomorphism to a path does not capture the additional edges that are allowed. Third, we give a block decomposition of the flattening of the all-different tensor: its rank at the cut $k \mid n-k$ is exactly $\binom{n}{k}$, and all its nonzero singular values coincide. The same decomposition extends to the full HAMPATH tensor: each block factors through the endpoint profiles of the paths in the two halves and a cut submatrix, which expresses its rank exactly.

The central methodological contribution is an *algebra of boundary signatures*: each subproblem is represented by the relation of boundary assignments that admit a satisfying extension; subproblems are combined by relational join, conjunction of local factors, and existential projection. In the weighted version, the same composition computes the number of extensions exactly. We prove the correctness of these operations and formulate a sufficient, explicitly conditional criterion for a compression by orbits or decision diagrams to yield an efficient algorithm. The framework recovers standard dynamic programming over tree decompositions and pinpoints where additional compression must occur: in the size of the intermediate signatures and in the cost of their operations.

As a structural case, we discuss dynamic programming for cographs and its limits. Experimental results are included as reproducible material in the package accompanying the manuscript, pending independent audit. We do not claim a general SAT solution or prove $P = NP$.

Keywords: SAT Equation; Hamiltonian path; graph isomorphism; symmetry; tensor rank; cographs; compilation; ETH.

1 Introduction

The SAT Equation associates with a CNF F over n variables an integer $S(F)$ of 2^n bits. The bit at position $\mathrm{pos}(x)$ is 1 if and only if the assignment x falsifies F; therefore F is unsatisfiable exactly when

$$S(F) = 2^{2^n} - 1.$$

The manuscripts of Riveros [1, 2, 3] study this encoding, its generalized form for arbitrary CNFs, and the queries it admits. The question of this note is not whether the encoding is correct —it is— but what can be compressed and what cannot.

Six observations organize the work.

1. **Symmetry.** If G preserves F, then $S(F)$ is constant on each orbit of G on $\{0,1\}^n$. The quotient has K bits, not 2^n. This compresses symmetric formulas a lot and rigid formulas not at all. The Burnside formula extends to signed permutations.
2. **Hardness of the orbit problem.** The general version of SAT-ORBIT is NP-complete; compression can only give a general algorithm under additional structural hypotheses.
3. **Pattern.** The Hamiltonian path is written as a condition on the superdiagonal of a permuted matrix. The condition leaves the other entries free; it is a subcube. Exact isomorphism to a path only recognizes graphs that already are a path.
4. **Rank.** The "all-different" factor has exact rank $\binom{n}{k}$ and a block decomposition with a flat singular spectrum; subset dynamic programming is close to that limit. The full tensor inherits a block decomposition: each block is the product of the endpoint profiles of the two halves and the cut submatrix, so its rank is exactly that of the submatrix.
5. **Structural compression.** Cographs admit a compact-state DP and path reconstruction. This is the first template for combining compressed solutions without expanding them.
6. **Encoding.** The subexponential algorithms of 2026 [4, 5] need compact encodings; an encoding with $O(n)$ variables is impossible in the worst case (for example, K_n) under an injective path–model representation, and one with $O(n)$ clauses remains open.

The note combines elementary theoretical results, exact experiments on small instances, and a review of recent manuscripts. Sections 2–5 present the three lenses and the positive cograph case; Sections 6–8 develop structural compression, the signature algebra, and encoding; Section 9 discusses open problems.

2 The SAT Equation and its quotient by symmetries

2.1 Definitions

Let F be a CNF over n variables and let $G = \mathrm{Aut}(F)$ be the group of variable permutations that preserves the set of clauses. The group acts on $\{0,1\}^n$ and hence on the positions of $S(F)$. Let K be the number of orbits and let T be the quotient vector:

$$T[O] = S(F)[x] \qquad \text{for any } x \in O.$$

The invariance of F guarantees that $T[O]$ is well defined.

Theorem 1. *If $G \le \mathrm{Aut}(F)$, then:*
(a) $S(F)$ is recovered from T by $S(F)[x] = T[\mathrm{orb}(x)]$;
(b) F is UNSAT if and only if T is all ones;
(c) if $T[O] = 0$, any $x \in O$ is a witness of satisfiability;
(d) the number of models is the sum of the sizes of the orbits with bit 0.

Proof. The invariance of F gives constancy of $S(F)$ on each orbit. The orbits partition $\{0,1\}^n$, so checking all of them is equivalent to checking all assignments. The count is the sum of the sizes of the orbits with bit 0. □

If all elements of G are known, the number of orbits is computed by Burnside:

$$K = \frac{1}{|G|} \sum_{\pi \in G} 2^{c(\pi)},$$

where $c(\pi)$ is the number of cycles of π on the variables. If G is large, one needs generators and group-theoretic algorithms.

2.2 Cost

Let T_G be the time to find the group, T_{orb} the time to enumerate the orbits, and m the number of clauses. The quotient costs

$$O(T_G + T_{\text{orb}} + K \cdot m)$$

in time and $O(K)$ bits in the representation. The full table costs $O(2^n \cdot m)$ and 2^n bits. The bound is useful when K is small; in the worst case G is trivial and $K = 2^n$.

2.3 Experiments

Table 1 summarizes exact instances. The times are not the point; the point is the reduction in the number of bits and evaluations.

instance	n	m	status	$\lvert G\rvert$	K	reduction
PHP(3,2)	6	9	UNSAT	12	13	4.9×
PHP(4,3)	12	22	UNSAT	144	87	47.1×
PHP(5,4)	20	45	UNSAT	2 880	1 053	995.8×
parity $n = 12$	12	2 048	SAT	479 001 600	13	315.1×
random 3-SAT $n = 8$	8	34	UNSAT	1	256	1.0×

Table 1: Quotient of the SAT Equation by the automorphism group. The $|G|$ column uses the certified subgroup; the full group could be larger.

In PHP(5,4), the quotient evaluates 1 053 representatives instead of 1 048 576 assignments; in the random 3-SAT there is no gain because the group is trivial.

The quotient can be computed without knowing the family. In a PHP(5,4) with renamed variables and shuffled clauses, a generic automorphism finder found the 2 880 elements of $S_5 \times S_4$ and Burnside returned $K = 1\,053$, without using the "PHP" label.

2.4 Signed symmetries

The 0/1 symmetry is not only a relabeling. Let $H_n = (\mathbb{Z}/2)^n \rtimes S_n$ be the hyperoctahedral group. An element (π, ε) acts on an assignment by

$$y[\pi(i)] = x[i] \oplus \varepsilon_i.$$

On literals, variable i goes to $\pi(i)$ and the sign is multiplied by $(-1)^{\varepsilon_i}$. Let $\text{Aut}^{\pm}(F)$ be the subgroup of H_n that preserves the set of clauses; the ordinary group is the subgroup with $\varepsilon = 0$. Theorem 1 applies to $\text{Aut}^{\pm}(F)$, and the quotient can be finer. Table 2 compares both quotients.

instance	n	signed aut.	ordinary K	signed K
$F = (x_1)$	2	2	4	2
$F = x_2$	2	2	4	2
odd parity	5	1 920	6	2
odd parity	6	23 040	7	2
PHP(3,2)	6	12	13	13
random 3-SAT	6	1	64	64

Table 2: Ordinary quotient versus the quotient by signed symmetries.

Parity wins because the signed group preserving parity is transitive inside each parity class. $F = (x_1)$ wins because it can flip the absent variable x_2. PHP and the random formula do not have that symmetry.

Theorem 2 (Fixed points of a signed permutation)**.** *For $g = (\pi, \varepsilon) \in H_n$, let $s_\varepsilon(C)$ be the XOR parity of the signs in a cycle C of π. Then*

$$|\operatorname{Fix}(g)| = \begin{cases} 2^{c(\pi)}, & s_\varepsilon(C) = 0 \text{ for every cycle } C, \\ 0, & \text{otherwise.} \end{cases}$$

Proof. A fixed assignment satisfies $x_{\pi(i)} = x_i \oplus \varepsilon_i$. Iterating around a cycle $C = (i_1 \ldots i_\ell)$ gives

$$x_{i_1} = x_{i_1} \oplus \varepsilon_{i_1} \oplus \cdots \oplus \varepsilon_{i_\ell}.$$

If the parity of the cycle is one, there is no solution. If it is zero, the first bit of the cycle is chosen freely and the rest are determined. Each compatible cycle contributes two independent choices. □

Corollary 3 (Signed Burnside)**.** *If $G \leq H_n$, the number of orbits of G on $\{0,1\}^n$ is*

$$K_G = \frac{1}{|G|} \sum_{g \in G} |\operatorname{Fix}(g)|,$$

with $|\operatorname{Fix}(g)|$ given by Theorem 2. For $\varepsilon = 0$ one recovers the ordinary formula.

The corollary counts orbits once the group is known; by itself it does not produce an efficient enumeration of representatives. That is the distinction in the next subsection.

The conclusion of this section is twofold. Compression by symmetry is real and can be exponential on structured families; it is not a general method, because a rigid formula has no group to exploit.

2.5 Orbit queries

Compressing the representation is not the same as solving SAT faster. If each bit of the quotient is obtained by evaluating the formula, one still has to visit K representatives. It is convenient to separate three tasks: finding the group, representing the orbits, and exploiting them. Burnside counts K but does not produce the representatives; orbit generation is a separate cost (T_{orb}), performed in the experiments with union-find and, for large groups, with stabilizer chains.

The natural algorithmic question is

$$\text{SAT-ORBIT}(F, G): \quad \exists O \in \{0,1\}^n/G \text{ with } T[O] = 0,$$

that is, deciding whether some orbit contains a satisfying assignment without enumerating all orbits.

Theorem 4 (Hardness of general SAT-ORBIT)**.** *The general version of SAT-ORBIT, with F and a group G that preserves F as input, is NP-complete.*

Proof. Membership in NP is immediate: a witness is an assignment x with $F(x) = 1$. For hardness, given a CNF F, take $G = \{\text{id}\}$. The trivial group preserves F, and there is an orbit with a model if and only if F is satisfiable. The transformation is polynomial. □

The theorem does not rule out efficient algorithms on restricted classes: it shows that compression by orbits needs additional structural hypotheses. It is useful to distinguish the syntactic group

$$\text{Aut}_{\text{synt}}(F) = \{g : g \text{ preserves the set of clauses}\}$$

from the semantic group

$$\text{Aut}_{\text{sem}}(F) = \{g : F(gx) = F(x) \text{ for every } x\}.$$

Always $\mathrm{Aut}_{\mathrm{synt}}(F) \leq \mathrm{Aut}_{\mathrm{sem}}(F)$ under a fixed normalization; the semantic group can be larger, but computing it can be as hard as deciding Boolean equivalences. The syntactic group is the conservative, verifiable option.

Characterizing the families of CNFs for which K is polynomial, the group admits polynomial generators, and satisfiability is decided without examining all representatives remains the central open problem of this line.

3 HAMPATH as an orbit–subcube intersection

3.1 The matrix formulation

Let G be a graph with adjacency matrix A. A permutation π of the vertices leaves the first superdiagonal full when

$$A_{\pi(1),\pi(2)} = A_{\pi(2),\pi(3)} = \cdots = A_{\pi(n-1),\pi(n)} = 1.$$

This is exactly the existence of a Hamiltonian path; for a cycle one adds $A_{\pi(n),\pi(1)} = 1$.

Let L_n be the set of adjacency matrices of simple graphs with a full superdiagonal; the other entries above the diagonal are free. L_n is a subcube with

$$2^{\binom{n}{2}-(n-1)}$$

elements. The orbit of A under conjugation by permutations is

$$\mathrm{Orb}(A) = \{P_\pi A P_\pi^{-1} : \pi \in S_n\}.$$

Then

$$G \text{ has a Hamiltonian path} \iff \mathrm{Orb}(A) \cap L_n \neq \varnothing.$$

This is an *orbit–subcube intersection*, not a graph isomorphism. Exact isomorphism solves $\mathrm{Orb}(A) \cap \{B\} \neq \varnothing$ for a single target B, or $\mathrm{Orb}(A) \cap \mathrm{Orb}(B) \neq \varnothing$ for all labelings of B.

3.2 Why exact isomorphism to a path is not enough

The temptation is to compare G with P_n. That only recognizes graphs that *are* a path. Elementary counterexamples:

- C_5 has a Hamiltonian path, but is not isomorphic to P_5;
- K_5 has a Hamiltonian path, but is not isomorphic to P_5;
- P_5 with a chord has a Hamiltonian path, but is not isomorphic to P_5.

Among the $2^{10} = 1\,024$ labeled graphs on 5 vertices, 633 have a Hamiltonian path; 573 of them are not isomorphic to P_5, and the other 60 are the labelings of P_5. The difference is the extra edges: L_n allows them, P_n does not.

3.3 Filters and the cubic case

The direct reduction enumerates L_n. One can filter by invariants of G: number of edges, degree sequence, spectrum, color refinement, etc. Table 3 shows how much the two simplest filters help.

The degree filter alone does not give a polynomial reduction; the other filters are not analyzed here in the same detail. In particular, every cubic graph has the same degree sequence $(3, \ldots, 3)$. Fix a cycle C_n and vary a perfect matching that uses no cycle edge; each choice gives a cubic graph with a Hamiltonian cycle. In blocks of six consecutive vertices there are at least two valid matchings, so for n a multiple of 6 there are at least $2^{n/6}$ choices. For a non-Hamiltonian cubic graph G of large n, the degree filter leaves those targets, and none is isomorphic to G.

It is also worth recalling two group-theoretic facts: general graph isomorphism has a quasi-polynomial algorithm [6], and bounded-degree graph isomorphism is polynomial [7]. Hamiltonian

graph	total targets	edge filter	degree filter
P_5	64	1	1
C_5	64	6	1
K_5	64	1	1
P_5 + chord $(1,3)$	64	6	4
random $n = 6$	1 024	120	2
random $n = 7$	32 768	5 005	22

Table 3: Targets in L_n that survive necessary filters. The two random graphs use seed 1.

path in cubic graphs remains NP-complete. Therefore a polynomial reduction from cubic HAM-PATH to bounded-degree GI would imply $P = NP$; a reduction to general GI would imply that GI is NP-complete. The obstacle is not cubic GI: it is the path pattern, not the isomorphism of the graph.

3.4 Transition formulation

Another representation avoids the explicit permutation. One uses variables $x_{ij} \in \{0, 1\}$, where $x_{ij} = 1$ indicates that the path uses the transition $i \to j$, with

$$\sum_j x_{ij} \le 1, \qquad \sum_i x_{ij} \le 1.$$

One vertex has out-degree zero, another has in-degree zero, and the rest have both degrees equal to one. The degree constraints are local; the difficulty is to exclude disconnected cycles and enforce a single component. The three natural routes are cut separation, algebraic certificates, and quotienting by transition automorphisms. None gives a polynomial algorithm yet; the formulation is a research direction, not a result.

4 Tensor rank and the "all-different" factor

4.1 Direct tensor network

The sum of paths is written as

$$\sum_{v_1,\dots,v_n} \prod_{i=1}^{n-1} A_{v_i,v_{i+1}} \cdot [v_1, \dots, v_n \text{ all distinct}].$$

The adjacency factor is a chain; the "all-different" factor connects all positions. If it is represented as a direct tensor network, the primal graph is complete and variable elimination has width $n-1$. The cost is n^n entries, compared with $n \cdot 2^n$ states of subset dynamic programming [8].

The dynamic program already uses $\binom{n}{k}$ subsets and, with the endpoint, $k\binom{n}{k}$ states at level k. The "all-different" factor does not disappear; it is paid in width, in number of terms, or in state size.

4.2 Rank of the isolated factor

Let D_n be the tensor with $D_n(v_1, \dots, v_n) = 1$ if the v_i are all distinct and 0 otherwise. The following result is elementary and valid for every n, not only for the computed cases.

Theorem 5 (Exact block decomposition)**.** *For the cut $k \mid n-k$, let $M_{n,k}$ be the flattening of D_n. After deleting zero rows and columns and reordering by the set of symbols appearing in each*

tuple, $M_{n,k}$ is permutationally equivalent to a block-diagonal matrix with $\binom{n}{k}$ blocks, each equal to

$$J_{k!,\,(n-k)!}.$$

Consequently, over any field,

$$\operatorname{rank}(M_{n,k}) = \binom{n}{k}.$$

Over $\mathbb{R}$, the nonzero singular values are all $\sqrt{k!(n-k)!}$, with multiplicity $\binom{n}{k}$.

Proof. A nonzero row is an injective k-tuple; its support is a set $S \subseteq [n]$ of size k. A nonzero column has support $T \subseteq [n]$ of size $n-k$. The entry is one exactly when $S \cap T = \varnothing$, which is equivalent to $T = [n] \setminus S$. Therefore rows with support S interact only with columns of support $[n] \setminus S$. There are $k!$ orders for S and $(n-k)!$ for its complement, and all entries of the block are one. Distinct blocks have disjoint supports. Each all-ones block has rank one; summing over the $\binom{n}{k}$ blocks gives the rank. The nonzero singular value of $J_{a,b}$ is $\sqrt{ab}$; the blocks have disjoint supports, so the spectra add. □

Corollary 6 (Flat spectrum)**.** *The matrix $M_{n,k}$ has spectral condition number one on its image: all its nonzero singular values coincide. Moreover,*

$$\|M_{n,k}\|_F^2 = \binom{n}{k} k!(n-k)! = n!.$$

The following table verifies the theorem in the first cases.

n	k	rank	$\binom{n}{k}$
4	2	6	6
5	2	10	10
5	3	10	10
6	3	20	20
6	4	15	15

Table 4: Verification of the exact rank of the all-different tensor over $\mathbb{F}_{10^9+7}$.

The maximum is

$$\binom{n}{\lfloor n/2 \rfloor} \sim \frac{2^n}{\sqrt{\pi n/2}}.$$

Any tensor-train that separates the positions at $k \mid n-k$ needs at least that bond. Subset dynamic programming uses $\binom{n}{k}$ subsets and $k\binom{n}{k}$ states at level k, so it is at that limit up to the factor k.

4.3 Rank of the full tensor

The full tensor $H(v_1, \ldots, v_n)$ is 1 if the order is a Hamiltonian path of G. Its flattening rank depends on the graph; throughout this note, "rank of the full tensor" means this flattening rank, not the CP rank of the tensor:

The all-different rank is $\binom{6}{2} = 15$, $\binom{6}{3} = 20$. The full tensor can be smaller, equal, or larger depending on the graph and the cut. The consequence is important: *rank does not measure the difficulty of HAMPATH.* A graph with a single undirected Hamiltonian path has rank 2 (the two orientations), and finding that path is still HAMPATH. A low-rank decomposition describes the tensor once its entries are known; it does not solve the search.

graph	paths	rank $k=2$	rank $k=3$
P_6	2	2	2
C_6	12	12	12
K_6	720	15	20
prism	60	18	30
$K_{3,3}$	72	18	18

Table 5: Rank of the full HAMPATH tensor for $n = 6$, computed over $\mathbb{F}_{10^9+7}$. The "paths" column counts oriented paths, that is, vertex orders.

4.4 Block decomposition of the full tensor

The previous theorem describes the isolated factor. The full tensor admits an analogous decomposition, but the blocks are no longer all-ones matrices: each one factors through the paths of the two halves and the adjacency matrix.

Theorem 7 (Blocks of the full tensor)**.** *Let $G = (V, E)$ with $V = [n]$, and let H_G be the tensor of oriented Hamiltonian paths. Fix k and, for each $S \subseteq V$ of size k, write $T = V \setminus S$. Let P_S be the matrix with rows indexed by the orders of S and columns by the vertices of S,*

$$P_S(\sigma, u) = 1 \iff \sigma \text{ is a path in } G[S] \text{ ending at } u,$$

and let Q_S be the matrix with rows indexed by T and columns by the orders of T,

$$Q_S(v, \tau) = 1 \iff \tau \text{ is a path in } G[T] \text{ starting at } v.$$

Then the block B_S of the flattening $k \mid n-k$ is

$$B_S = P_S\, A[S, T]\, Q_S.$$

If

$$E_S = \{u \in S : P_S(\sigma, u) = 1 \text{ for some } \sigma\}, \qquad F_S = \{v \in T : Q_S(v, \tau) = 1 \text{ for some } \tau\},$$

then, over any field,

$$\operatorname{rank}(B_S) = \operatorname{rank}(A[E_S, F_S]) \le \min\{|E_S|, |F_S|, |\partial S|, k, n-k\},$$

where ∂S is the set of edges between S and T. Moreover, if $\operatorname{rank}(H_G)$ denotes the rank of the flattening $k \mid n-k$,

$$\operatorname{rank}(H_G) = \sum_{|S|=k} \operatorname{rank}(B_S).$$

In particular,

$$\operatorname{rank}(H_G) \le \sum_{|S|=k} \min\{|\partial S|, k, n-k\},$$

a bound that depends only on the cuts of G.

Proof. Fix S. A row of the block is indexed by an order σ of S, and a column by an order τ of T. The entry of H_G is one exactly when σ and τ are paths and the edge (σ_k, τ_1) belongs to E. Hence $B_S(\sigma, \tau) = A_{\sigma_k, \tau_1}$, which is the (σ, τ) entry of $P_S A[S, T] Q_S$.

The row space of P_S is $\langle e_u : u \in E_S \rangle$: the nonzero rows are canonical vectors and each $u \in E_S$ appears at least once. Analogously, the column space of Q_S is $\langle e_v : v \in F_S \rangle$. Restricting $A[S, T]$ to those two subspaces leaves $A[E_S, F_S]$, whence $\operatorname{rank}(B_S) = \operatorname{rank}(A[E_S, F_S])$. The bound follows because that submatrix has at most $|\partial S|$ nonzero entries, and from its dimensions.

The blocks B_S have disjoint row supports (each row belongs to a unique S) and disjoint column supports (each column belongs to a unique $T = V \setminus S$); after reordering, the flattening is block diagonal. The rank is the sum. □

Corollary 8 (Counting by halves)**.** *With* $h_S(u) = |\{\sigma : P_S(\sigma, u) = 1\}|$ *and* $g_S(v) = |\{\tau : Q_S(v, \tau) = 1\}|$,

$$\#\{\textit{oriented Hamiltonian paths of } G\} = \sum_{|S|=k} \sum_{u \in S} \sum_{v \in T} h_S(u)\, A_{uv}\, g_S(v).$$

Proof. The sum of all entries of B_S is $\sum_{u,v} h_S(u) A_{uv} g_S(v)$. Every oriented Hamiltonian path has a unique cut $k \mid n-k$, so it is counted exactly once. □

Observation 9. For $G = K_n$, every partial order is a path, $E_S = S$, $F_S = T$, and $A[S, T]$ is all ones; each block is $J_{k!,(n-k)!}$ and one recovers Theorem 5. For $G = P_n$, only a few profiles are nonzero and the total rank is two. For the prism on six vertices, the sum exceeds $\binom{6}{3} = 20$: the full tensor can have more rank than the all-different factor.

The following table verifies the decomposition for $n = 6$, cut $3 \mid 3$. The rightmost column is the rank of the full flattening, computed independently.

graph	paths	$\max_S \operatorname{rank}(B_S)$	$\sum_S \operatorname{rank}(B_S)$	$\operatorname{rank}(H_G)$
P_6	2	1	2	2
C_6	12	2	12	12
K_6	720	1	20	20
prism	60	3	30	30
$K_{3,3}$	72	1	18	18

Table 6: Blocks of the full HAMPATH tensor for $n = 6$ and $k = 3$, over $\mathbb{F}_{10^9+7}$. Paths are oriented. The total rank is the sum of the block ranks, and it coincides with the rank of the flattening computed directly.

The formula shows what controls the rank of the full tensor: the endpoint profiles of $G[S]$ and $G[T]$, and the cut submatrices. Both objects can be hard to compute; in particular, $E_S \neq \varnothing$ is exactly the existence of a Hamiltonian path in $G[S]$. Therefore low rank does not certify ease and high rank does not certify difficulty. What the decomposition identifies is the exact state of a composition by halves: one endpoint vector per half and a bilinear form given by the adjacency matrix.

5 A positive case: HAMPATH in cographs

Cographs are built from vertices with disjoint union and join (all edges between the two parts) [9]. For a cograph G, let $p(G)$ be the minimum number of vertex-disjoint (possibly trivial) paths that cover all vertices.

Lemma 10. *The set of achievable cover numbers is the interval* $[p(G), n]$.

Proof. From a minimum cover one can remove edges from the paths; each removed edge splits a path in two and raises the number by one. Repeating until only singletons remain reaches n. □

With the lemma, the dynamic program is immediate:

$$\begin{aligned} \text{leaf:} \quad & (p, n) = (1, 1), \\ G_1 \cup G_2: \quad & p = p_1 + p_2, \quad n = n_1 + n_2, \\ G_1 \vee G_2: \quad & p = \max\bigl(1, \delta([p_1, n_1], [p_2, n_2])\bigr), \quad n = n_1 + n_2, \end{aligned}$$

where, for intervals $[a, b]$ and $[c, e]$, the distance is

$$\delta([a, b], [c, e]) = \max\{0,\, a - e,\, c - b\}.$$

Theorem 11. *G has a Hamiltonian path if and only if $p(G) = 1$.*

Proof. In the disjoint union there are no edges between the parts, so paths cannot be merged. In the join, all cross edges are present. If $p_1 \geq p_2$, the p_2 paths of G_2 can be used as bridges to chain the paths of G_1; the resulting number is $\max(1, p_1 - p_2)$. The lemma allows any value in the intervals, and the minimum is obtained from the distance between them. □

The same argument is constructive.

Theorem 12 (Reconstruction)**.** *Given the composition tree and $p(G) = 1$, one can construct a Hamiltonian path of G in time $O(n^2)$ with explicit lists; the decision is $O(n)$.*

Proof. One constructs a cover with p paths by recursion. At a leaf, the only path is the vertex. At a union, the covers of the two parts are concatenated. At a join, one chooses p_1 and p_2 in the intervals and alternates the paths of the two parts using the cross edges; if paths remain on one side, they stay as separate components. The lemma guarantees that any count in the interval can be chosen, and if the count is smaller than desired one splits a path by removing an edge. For $p = 1$ the result is a Hamiltonian path. Each concatenation traverses lists of total size $O(n)$ and the composition tree has $O(n)$ nodes, whence $O(n^2)$. □

Given the composition tree, the DP is $O(n)$; cograph recognition is linear with known algorithms. The implementation `hampath_cograph.py` not only decides: it reconstructs the path. It was checked against Held–Karp on 700 random cotrees with $n = 2, \ldots, 8$; in the 294 cases with $p = 1$, every reconstructed path was validated, with no discrepancies.

The limit is clear: in the modular decomposition of a general graph, the *prime* nodes are not cographs and do not admit the interval DP; the method does not extend automatically, and HAMPATH may remain NP-complete on that class. The result solves a broad class, not all graphs.

6 Towards structural compression

6.1 The cograph template

The cograph DP not only decides HAMPATH: it is an example of an *operation that combines compressed states*. The state of a subgraph is the interval

$$I(G) = [p(G),\, n],$$

which summarizes the achievable cover numbers without enumerating the covers. Union and join combine intervals with constant-cost operations:

$$I(G_1 \cup G_2) = [p_1 + p_2,\, n_1 + n_2],$$

$$I(G_1 \vee G_2) = [\max(1, \delta([p_1, n_1], [p_2, n_2])),\, n_1 + n_2].$$

The decision is $O(n)$ and the explicit reconstruction is $O(n^2)$. The conceptual point is that the state is small and the composition does not expand the solutions.

6.2 The endpoint-profile template

Theorem 7 gives a second template, this time for general graphs. The state of a subset S is the profile

$$h_S(u) = \#\{\text{paths in } G[S] \text{ ending at } u\},$$

and the combination of two halves S, T is the bilinear form

$$\sum_{u \in S} \sum_{v \in T} h_S(u)\, A_{uv}\, g_T(v),$$

where $g_T(v)$ counts the paths of $G[T]$ starting at v. The operation is exact and does not expand the paths. The difference with cographs is the size of the state: h_S has $|S|$ components and computing it contains the decision of HAMPATH on $G[S]$. In cographs, the relevant state collapses to the interval $[p, n]$. The open question is which graph classes admit a profile of polynomial size that is computable in polynomial time.

6.3 The program

The question organizing the program is whether there are classes of CNFs (or graphs) with a decomposition into subproblems, a local state computable in polynomial time, and a polynomial-time combination operation that decides SAT or HAMPATH without enumerating orbits or assignments.

The candidate ingredients are:

1. modular or tree decomposition;
2. computation of local automorphisms;
3. compact representation of orbits or covers;
4. combination by dynamic programming;
5. verifiable certificates of satisfiability or unsatisfiability.

The critical condition is not that the final representation be small, but that the number of states and the cost of combining them be polynomial. Cographs are the first template; the open question is which state replaces the interval $[p, n]$ when prime nodes appear, or when the problem is SAT rather than HAMPATH.

This program does not prove $P = NP$. It proposes a concrete direction: find an algebraic operation that combines compressed solutions of subproblems without expanding them, and enlarge the class of formulas to which it applies.

7 Algebra of boundary signatures: composition without expanding models

Compression of a solution family is algorithmically useful only if it can be preserved when subproblems are combined. This section formalizes an exact operation for doing so. The basic operation is not a heuristic: it is the relational join followed, when appropriate, by existential projection. It is standard dynamic programming over tree decompositions; the specific contribution is the compression criterion and compatibility with symmetries, not the correctness of the join. The possibility of maintaining a small representation after these operations is a separate question and constitutes the research problem.

7.1 Relational signatures

Let $F = \bigwedge_{a \in \mathcal{A}} f_a$ be a Boolean formula, where each factor f_a depends on a set of variables $\mathrm{var}(f_a)$. For a set of boundary variables B, and a set of factors $\mathcal{A}_t$ assigned to a subproblem, write $V_t = \bigcup_{a \in \mathcal{A}_t} \mathrm{var}(f_a)$. The *boundary signature* of the subproblem is the relation

$$R_t = \left\{ b \in \{0,1\}^{B \cap V_t} : \exists u \in \{0,1\}^{V_t \setminus B} \bigwedge_{a \in \mathcal{A}_t} f_a(b \cup u) = 1 \right\}.$$

Thus R_t contains exactly the boundary assignments that admit a satisfying extension inside the subproblem. The relation can be represented explicitly as a table, or implicitly by a BDD, ZDD, Boolean circuit, or another canonical structure.

For relations $R \subseteq \{0,1\}^{X \cup S}$ and $Q \subseteq \{0,1\}^{Y \cup S}$, where the shared variables are S, define the natural join

$$R \bowtie_S Q = \{ z \in \{0,1\}^{X \cup Y \cup S} : z|_{X \cup S} \in R,\ z|_{Y \cup S} \in Q \}.$$

For $Z \subseteq W$, existential projection is

$$\pi_Z(R) = \{z \in \{0,1\}^Z : \exists w \in R,\ w|_Z = z\}.$$

In a tree decomposition, before the join both relations are extended to the common boundary of the node; variables that appear only in one child are introduced without quantification. Thus the schemas coincide exactly on the shared variables. The composition of subproblems is obtained by taking the join of their signatures and projecting the variables that leave the boundary.

Lemma 13 (Composition laws)**.** *The relational join is associative (with shared-variable restrictions interpreted in the natural way), and existential projection satisfies*

$$\pi_X(\pi_Y(R)) = \pi_X(R) \qquad \text{if } X \subseteq Y.$$

Moreover, if two subproblems have disjoint internal variables and share only S, then their joint signature on the remaining boundary is exactly the projection of the join of their signatures.

Proof. Membership in a join is equivalent to the conjunction of the membership conditions of each relation. Conjunction is associative, which proves the first claim. The double projection expresses two successive existential quantifications; by associativity of $\exists$, it is equivalent to quantifying directly all variables outside X. For the last claim, a boundary assignment extends to a solution of the joined subproblem if and only if there are internal extensions for each child that coincide on the shared variables. That is precisely the membership condition of the join followed by the projection. □

7.2 Weighted version for #SAT

The relational signature decides existence. Exact model counting is $\#P$-complete in general [10]; the DP is efficient only when the intermediate signatures are small. To count extensions, one uses a boundary function

$$W_t : \{0,1\}^B \longrightarrow \mathbb{N},$$

where $W_t(b)$ is the number of assignments to the internal variables of the subproblem that, together with b, satisfy all its factors. If two children have disjoint internal variables conditioned on the same boundary B, the combination at a union node is

$$W_t(b) = W_{t_1}(b)\, W_{t_2}(b).$$

When a variable v leaves the boundary, it is eliminated by summing its two values:

$$W_{\text{parent}}(b) = W_{\text{child}}(b[v \leftarrow 0]) + W_{\text{child}}(b[v \leftarrow 1]).$$

Introducing a boundary variable extends the function without summing. These are the sum-product operations over the semiring $(\mathbb{N}, +, \cdot, 0, 1)$. For SAT decision one can work in the Boolean semiring $(\{0,1\}, \vee, \wedge, 0, 1)$.

Theorem 14 (Correctness of signature composition)**.** *Let $(T, (B_t)_{t \in T})$ be a tree decomposition of the primal graph of F (variables joined when they share a factor), and assign each factor to a bag containing its scope. We may assume without loss of generality that it is nice (leaf, introduction, forget, and join nodes), with a constant factor of additional bags. The dynamic program that stores in each bag the relational signature defined above, with the conjunction of the local factors incorporated into the initial signature of the bag, and combines by join at union nodes and by existential projection at forget nodes, correctly decides whether F is satisfiable. In the weighted version, replacing join by multiplication and forget by sum computes exactly $\#F$.*

Proof. Proceed by induction from the leaves to the root. The invariant is that an entry of bag B_t belongs to the signature if and only if it admits an extension to all internal variables of the subtree of t that satisfies all factors assigned there. At a leaf, the initial signature is the conjunction of the local factors assigned to the bag. At an introduction, the domain of the boundary is enlarged without quantifying the variable. At a forget, the variable leaving the boundary is existentially quantified. At a union, the subtrees do not share internal variables and their restrictions couple exactly in the bag; by Lemma 13, the join preserves the invariant. At the root, the boundary can be emptied and the resulting relation is nonempty if and only if a model exists. For the weighted version, independent extensions multiply and the alternatives of a forgotten variable are summed, counting each assignment exactly once. □

7.3 Compression by representations and boundary symmetry

The explicit table of a signature on a boundary of size b has 2^b entries. The previous theorem guarantees correctness, not compression. The signature can be stored in an implicit representation, for example a reduced ordered BDD, and the above operations implemented by Boolean operations and quantification. However, the size of a BDD can be exponential for certain orders and functions; we do not claim that a polynomial representation exists for every CNF.

Symmetry offers an additional reduction when it acts on the boundary. Let $H \leq S_B$ be a group that preserves the signature R_t, that is, $b \in R_t$ if and only if $h \cdot b \in R_t$ for every $h \in H$. Then R_t is a union of orbits and can be stored as a set of orbits. For weighted signatures one additionally requires $W_t(h \cdot b) = W_t(b)$. Compression is valid for a composition operation only when the actions of the children and the parent are compatible and the operations are equivariant; it is not enough for each child to have some different symmetry.

Proposition 15 (Stability of invariance under composition)**.** *Let G be a group acting on the assignments of a boundary B. If the Boolean signatures $R_1, R_2 \subseteq \{0,1\}^B$ are G-invariant, then $R_1 \cap R_2$ is also G-invariant. If $W_1, W_2 : \{0,1\}^B \to \mathbb{N}$ are G-invariant, then the pointwise product $W_1 W_2$ is G-invariant. More generally, let $\phi : \{0,1\}^B \to \{0,1\}^C$ be an equivariant map with respect to actions of G on B and C. Then*

$$\widetilde{W}(c) = \sum_{\phi(b)=c} W(b)$$

is invariant under the induced action of G on C, provided W is G-invariant.

Proof. For the intersection and the product, use the invariance of each signature and each function directly. For the projection, equivariance implies that $b \mapsto g \cdot b$ is a bijection between the fibers $\phi^{-1}(c)$ and $\phi^{-1}(g \cdot c)$. Since W is constant on orbits, the sums over both fibers coincide. □

The proposition gives a precise algebraic condition for orbit compression to survive transitions: the symmetries must act compatibly on the boundary and on the forget map. In that case one can store one value per orbit instead of one value per boundary assignment. This does not eliminate the cost of finding generators, computing orbits, or producing representatives; those costs must be included in any algorithmic analysis.

Proposition 16 (Conditional criterion for a compressed algorithm)**.** *Consider a tree decomposition with q bags. Suppose that for each bag t:*

(a) the exact signature admits a representation with at most K_t canonical states;
(b) the introduction, forget, and union operations can be computed directly on those representations, without expanding the $2^{|B_t|}$ assignments;
(c) the cost of each operation is bounded by a fixed polynomial in the sizes of the input and output representations;

(d) the initial signature of each bag (the conjunction of its local factors) is constructed in time polynomial in $|F|$ and K_t.

Then SAT is decided in time

$$\operatorname{poly}\left(|F|, q, \max_t K_t\right),$$

and the weighted version computes #SAT with polynomial complexity in those parameters and in the length of the intermediate integers.

Proof. The dynamic program processes each bag once. By hypothesis, each transition is computed in time polynomial in the sizes of its representations; there are $O(q)$ transitions in a binary tree decomposition, and by (d) constructing the initial signatures costs time polynomial in $|F|$ and K_t. Combining these bounds gives the result. In the weighted case one must include the binary length of the counts, which can be $O(n)$ bits for the total count. □

The proposition is a sufficient criterion, not a construction of the representations in item (a). For a tree decomposition of width w, the explicit table gives $K_t \le 2^{w+1}$, recovering the standard $2^{O(w)}\operatorname{poly}(|F|)$ algorithm. The improvement sought here is to obtain small K_t even when the boundary is large, for example through local symmetries, small BDDs, or additional algebraic structure.

7.4 Minimal example: separation by a separator

Let X, S, Y be disjoint sets of variables and

$$F(X, S, Y) = F_1(X, S) \wedge F_2(S, Y).$$

Define

$$R_1 = \{s : \exists x\ F_1(x, s)\}, \qquad R_2 = \{s : \exists y\ F_2(s, y)\}.$$

Then

$$F \text{ is satisfiable} \iff R_1 \cap R_2 \neq \varnothing.$$

One does not enumerate the assignments to $X \cup S \cup Y$: each side transmits only the information about S that affects the other side. To count models, define $W_1(s) = |\{x : F_1(x, s)\}|$ and similarly W_2; then

$$\#F = \sum_{s \in \{0,1\}^S} W_1(s) W_2(s).$$

This example is elementary, but it contains the central operation of the program: summarize each side by its behavior on the boundary and combine the summaries, not the full solutions.

7.5 Derived research questions

The framework leaves three concrete problems:

1. **Signatures with effective symmetry.** Identify classes of CNFs where the orbits of boundary assignments are few and computable from generators, and where the transitions are equivariant.
2. **Canonical compression.** Find a class of representations (automata, BDDs, deterministic circuits, or algebraic modules) closed under join and projection, with size bounds for a nontrivial family of CNFs.
3. **Prime nodes.** In modular decompositions of graphs or variable decompositions, characterize what boundary information a prime node needs to combine paths or models without enumerating its internal solutions.

The long-term goal is to prove uniform bounds for K_t and for the operations, not merely to observe that some instances compress. If a polynomial representation for arbitrary CNFs were obtained under polynomial construction, a polynomial algorithm for SAT would follow; this manuscript does not construct such a representation.

8 Encoding and the 2026 subexponential algorithms

8.1 The manuscripts

The preprints of OpenAI [4] and Anthropic [5] give deterministic subexponential algorithms for #SAT. The Anthropic one uses a tensor network of clauses, cuts, merging, and the eight-square identity; the OpenAI one uses sparsification, polynomial interpolation, and shared linear summaries. According to their results, ETH and SETH are refuted. Neither implies $P = NP$ or gives practical algorithms. The Lean formalization of the Anthropic manuscript verifies a formal statement; checking that the statement faithfully represents the mathematical theorem is a separate task, and it is not used here as a certificate that the formal statement corresponds to the theorem.

The two works are recent preprints and their conclusions should be cited as results announced in those manuscripts, not as an independent review performed here. The explicit theorem of the OpenAI preprint gives, for fixed k,

$$(n + M)^{O_k(1)} 2^{O_k(n \log\log n/\sqrt{\log n})},$$

and for $M \le Dn$ the bound $2^{O_{k,D}(n/\sqrt{\log n})}$, with polynomial space. The abstract of the preprint announces the form $2^{n/(\log n)^{1/2+o(1)}}$, with $\rho(n) = n(\log\log n)^2/\sqrt{\log n} = n/(\log n)^{1/2+o(1)}$, which does not coincide literally with the explicit theorem and is weaker; the theorem gives the stronger bound. The scope note of the OpenAI Lean formalization formalizes $2^{O_k(n(\log\log n)^2/\sqrt{\log n})}$, also weaker than the explicit theorem, and it differs from the quantitative version of the theorem. For this reason we cite the explicit theorem bound and do not use the abstract version as established [4, 11].

The Anthropic preprint announces a deterministic subexponential algorithm for #3-SAT and a Lean 4 formalization [5]. In both cases, a $2^{o(n)}$ bound for k-SAT would refute ETH if the theorems and their computational models are verified as stated. It does not imply $P = NP$, it does not by itself imply subexponential time for every NP problem under general polynomial reductions, and it is not a practical solver. The exact correspondence between the mathematical statement, the formal model, and the implementation must be evaluated separately. We do not use these results as premises of any theorem in this article.

8.2 Encoding HAMPATH

It is useful to separate four parameters: N variables, M clauses (the m of the previous sections), L description bits, and k clause width. The bounds of [4, 5] depend on N and M; total input size and width are not interchangeable. In what follows, whenever an encoding is compared, we indicate which of the four parameters is being measured.

If each path requires a distinct model, in the worst case (for example, K_n) a permutation needs $\Omega(n \log n)$ Boolean variables: there are $n!$ paths and a CNF with N variables has at most 2^N assignments. An encoding with $O(n)$ variables and shared models is not ruled out by this argument; what is ruled out is the injective path–model representation.

An encoding with $O(n)$ *clauses* is another matter; the Anthropic bound depends on M and on the width k. The flow encoding for cubic graphs is 3-CNF ($k = 3$) and uses $N = O(n \log n)$ variables and $M = O(n \log n)$ clauses. The Anthropic theorem gives

$$2^{O(\min\{k^2 N \log\log N/\sqrt{\log N},\, kM/\sqrt{\log M}\})}.$$

The construction chooses a super-root, adds both arcs of each edge, and uses binary flow variables to enforce a single component; the degree and flow constraints give that size. A Hamiltonian path is directed from the super-root with decreasing flow; conversely, the flow and the degree constraints exclude disconnected cycles. With the explicit bounds it does not improve on 2^n: the bound in M gives $2^{O(n\sqrt{\log n})}$ and the bound in N gives $2^{O(n\sqrt{\log n}\log\log n)}$. To improve with the bound in M, one would need $M = O(n)$.

8.3 Linear encodings that do exist

Set Splitting with bounded set size $k = O(1)$ has a linear encoding: one variable per element and two clauses per set (at least one 0 and at least one 1). Verification found no discrepancies. With $M = O(N)$ and $k = O(1)$, the Anthropic bound in M gives $2^{O(N/\sqrt{\log N})}$, subexponential.

Coloring with fixed k is also linear: $N = nk$, $M = O(nk^2 + ek)$, where e is the number of edges, and verification found no discrepancies. If $e = O(n)$, then $M = O(N)$ and the bound in M gives $2^{O(N/\sqrt{\log N})}$, subexponential in N.

problem	variables	clauses	consequence
Set Splitting	N	$2 \cdot$ sets	subexp. if $M = O(N)$, $k = O(1)$
Coloring (fixed k)	nk	$O(nk^2 + ek)$	subexp. if $e = O(n)$
HAMPATH cubic flow ($k = 3$)	$O(n \log n)$	$O(n \log n)$	no explicit improvement
HAMPATH (path model)	$\Omega(n \log n)$	$O(n)$ open	subexp. if it exists with constant width

Table 7: Encoding sizes and consequence with the 2026 bounds.

9 Discussion and open problems

The resulting map is as follows. Symmetry compresses the SAT Equation on structured families and not on rigid formulas. The Hamiltonian path is not a graph isomorphism problem: it is an orbit–subcube intersection, and exact isomorphism to a path only solves the case without extra edges. The "all-different" factor has high rank and subset dynamic programming is close to that limit. The rank of the full tensor decomposes into endpoint profiles and cut submatrices; it also does not measure difficulty, because the profiles encode HAMPATH decisions on subgraphs.

Several concrete problems remain:

1. **HAMPATH with $O(n)$ clauses.** The cubic flow encoding gives $O(n \log n)$. If it is lowered to $O(n)$ with constant clause width, the Anthropic bound in M gives $2^{O(n/\sqrt{\log n})}$.
2. **Better crossing decomposition.** In the crossing-decomposition recursion of the Anthropic algorithm, the constant is $\rho = 11/12$. Lower it to 0.905 or less with an explicit, verifiable identity.
3. **The OpenAI abstract discrepancy.** The abstract announces $2^{n/(\log n)^{1/2+o(1)}} = 2^{\rho(n)}$, with $\rho(n) = n(\log\log n)^2/\sqrt{\log n}$, and the explicit theorem gives $2^{O(n \log\log n/\sqrt{\log n})}$; they do not coincide literally and the abstract is weaker. We use the explicit theorem bound, which is stronger.
4. **Cographs to modular decomposition.** Extend the interval DP to prime nodes of bounded structure.
5. **Compact endpoint profiles.** Theorem 7 expresses the full tensor through profiles h_S. Which graph classes admit a profile of polynomial size that is computable in polynomial time?
6. **Q-ary generalization.** Adapt the Boolean algorithms to variables of domain n, avoiding the cost of $\log n$ bits.
7. **Signed symmetries at larger scale.** Find the signed group and its orbits without enumerating $2^n n!$.
8. **Gadgets and joint invariants HAMPATH $\to$ GI.** No reduction is known; the cubic case shows that a reduction to bounded-degree GI (the same cubic graph) would imply $P = NP$, and a reduction to general GI would imply that GI is NP-complete. One needs invariants that certify $\mathrm{Orb}(A) \cap L_n \neq \varnothing$, not only comparison with P_n.

The underlying question is whether there is a polynomial representation of the model set of SAT, that is, a description of $\mathcal{S}(F) = \{x : F(x) = 1\}$ as a union of few compact classes, with a polynomial procedure to decide whether some class is nonempty. The quotient by symmetries

is a particular case of that idea; the difficulty is to construct the representation, not to prove that it exists.

None of these points implies $P = NP$ by itself. The value of the note is the map: it separates real compressions from those that only change the name of the problem.

10 Status of experimental claims

The tables in the sections on symmetry, matrix filters, rank, the full tensor, full-tensor blocks, and cograph reconstruction were reproduced with the scripts, seeds, and outputs of the accompanying package. The package folder is `paper_material`. An independent review must still verify the figures and the execution environment. The package includes: (i) source code and version; (ii) instances and seeds; (iii) representation and normalization conventions; (iv) execution instructions; and (v) the obtained outputs. The figures are presented as reproducible evidence, not as premises of the theorems.

In particular, automorphism-group tables must state whether the full group or only a certified subgroup is computed; rank tables must specify the field and the order of rows and columns; and HAMPATH tables must distinguish directed, undirected, and permutation-equivalent paths. Until an independent audit verifies it, the figures are reproducible evidence and not premises of the theoretical results.

11 Reproducibility

The `paper_material/` package contains the scripts and outputs. The `scripts/` folder includes:

- `sat_equation.py`: generalized equation, DIMACS, verification.
- `sat_symmetry.py`, `signed_symmetry.py`: ordinary and signed quotients.
- `generic_k.py`: K from an obfuscated CNF.
- `hamiltonian_matrix.py`, `hampath_to_gi.py`: matrix formulation and reduction to GI.
- `hampath_tensor.py`: HAMPATH tensor.
- `hampath_rank.py`, `hampath_full_rank.py`: rank.
- `hampath_block_rank.py`: blocks of the full tensor.
- `hampath_cograph.py`: DP and reconstruction in cographs.
- `linear_encodings.py`: Set Splitting and coloring.
- `eth_improvements.py`: eight-square identity and bounds.

Outputs are in `outputs/` and DIMACS instances in `data/instances/`. The environment is Python 3 with the standard library, with no external dependencies. The seeds are embedded in the scripts; the exact commands are in the package `README.md`. Times should not be the only measure: bits, states, diagram sizes, and number of evaluations must also be reported. An independent audit must repeat the runs before citing the figures in a publication.

References

[1] O. Riveros. *Consultas comprimidas sobre la Ecuación SAT: bit, rank, select y oráculos de conteo para CNF arbitrarias* [Compressed queries on the SAT Equation: bit, rank, select, and counting oracles for arbitrary CNFs]. Obras Completas, Vol. I, 2026.

[2] O. Riveros. *La Ecuación SAT generalizada* [The generalized SAT Equation]. Manuscript, 2025.

[3] O. Riveros. *Complejidad de representaciones para la Ecuación SAT: cotas superiores, inferiores y compilación híbrida* [Representation complexity for the SAT Equation: upper bounds, lower bounds, and hybrid compilation]. Obras Completas, Vol. I, 2026.

[4] OpenAI. *The Exponential Time Hypothesis is False.* Preprint, October 17, 2026, revised October 25, 2026. https://github.com/openai/math/tree/main/preprints/The-Exponential-Time-Hypothesis-is-False-October-25-2026.

[5] Claude (Anthropic). *A $2^{o(n)}$ Time Algorithm for 3-SAT.* Preprint, October 26, 2026. https://hexagonmath.org/2610.00577v1.

[6] L. Babai. Graph isomorphism in quasipolynomial time. In *Proceedings of the 48th Annual ACM Symposium on Theory of Computing*, pages 684–697, 2016.

[7] E. M. Luks. Isomorphism of graphs of bounded valence can be tested in polynomial time. *Journal of Computer and System Sciences*, 25(1):42–65, 1982.

[8] M. Held and R. M. Karp. A dynamic programming approach to sequencing problems. *Journal of the Society for Industrial and Applied Mathematics*, 10(1):196–210, 1962.

[9] D. G. Corneil, H. Lerchs, and L. Stewart Burlingham. Complement reducible graphs. *Discrete Applied Mathematics*, 3(3):163–174, 1981.

[10] L. G. Valiant. The complexity of enumeration and reliability problems. *SIAM Journal on Computing*, 8(3):410–421, 1979.

[11] OpenAI. *The Exponential Time Hypothesis is False.* Lean formalization scope note. OpenAI Math repository, 2026. https://github.com/openai/math/blob/main/lean/docs/378.md.